Starlette Server

Starlette server using **Uvicorn**.

1. Install the required packages

pip install starlette uvicorn fastapi

2. Create app.py

```
from starlette.applications import Starlette
from starlette.responses import JSONResponse
from starlette.routing import Route

async def homepage(request):
    return JSONResponse({"message": "Hello, Starlette!"})

app = Starlette(routes=[
    Route("/", homepage)
])
```

3. Start the server

uvicorn app:app --reload

Where:

- app (before :) = the Python filename (app.py)
- app (after :) = the Starlette application object
- --reload = automatically restarts the server when files change

4. Open in your browser

http://127.0.0.1:8000/

Expected response:

```
{
  "message": "Hello, Starlette!"
}
```

Other commands:

Run on a different port:

uvicorn app:app --reload --port 5000

Run on all network interfaces:

uvicorn app:app --host 0.0.0.0 --port 8000 --reload

Run with multiple workers (production):

uvicorn app:app --host 0.0.0.0 --port 8000 --workers 4

Starlette vs FastAPI

FastAPI is built **on top of Starlette**.

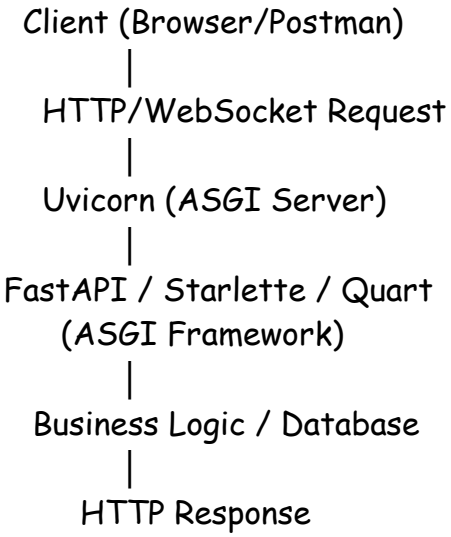
Starlette provides the ASGI framework, routing, middleware, and request/response handling, while FastAPI adds API-focused features such as automatic validation, dependency injection, and OpenAPI documentation.

What is an ASGI Framework?

ASGI (Asynchronous Server Gateway Interface) is the standard interface between **Python web applications/frameworks** and **web servers**. It is the successor to WSGI and was designed to support asynchronous programming.

An ASGI framework lets application handle many client requests concurrently without blocking, making it well suited for modern web applications, APIs, WebSockets, and real-time services.

ASGI Architecture



What does ASGI stand for?

- **A** – Asynchronous
- **S** – Server
- **G** – Gateway
- **I** – Interface

It defines how a Python application communicates with an ASGI server.

Feature	Starlette	FastAPI
Purpose	Lightweight ASGI framework	High-level API framework
Built On	ASGI	Starlette
Route Declaration	Route() objects	@app.get(), @app.post() decorators
Data Validation	Manual	Automatic using Pydantic

Feature	Starlette	FastAPI
Type Hints	Optional	Core feature
OpenAPI/Swagger	✗ Not built in	✓ Automatic
ReDoc	✗	✓ Automatic
Dependency Injection	✗	✓ Built-in
Request Parsing	Manual	Automatic
Response Models	✗	✓
Authentication Utilities	Manual	Rich ecosystem
Middleware	✓	✓ (inherits Starlette)
Background Tasks	✓	✓
WebSockets	✓	✓
Performance	Excellent	Excellent (very similar)
Learning Curve	Lower-level	Easier for API development

Architecture

```
Your Application
      |
FastAPI Framework
      |
Starlette Framework
      |
ASGI Server (Uvicorn)
      |
Operating System
```

FastAPI uses Starlette internally for:

- Routing
- Middleware
- Request/Response objects
- Background tasks
- WebSockets
- Lifespan events

Creating an Application

Starlette

```
from starlette.applications import Starlette

app = Starlette()
```

Routes are registered explicitly:

```
from starlette.routing import Route

async def home(request):
    return JSONResponse({"message": "Hello"})

routes = [
    Route("/", home)
]

app = Starlette(routes=routes)
```

FastAPI

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def home():
    return {"message": "Hello"}
```

This decorator-based style is generally more concise.

Reading JSON Data

Starlette

```
async def create_customer(request):
    data = await request.json()
```

You manually parse and validate the request body.

FastAPI

```
from pydantic import BaseModel

class Customer(BaseModel):
    name: str
    email: str

@app.post("/customers")
async def create_customer(customer: Customer):
    return customer
```

FastAPI automatically:

- Parses JSON
- Validates types
- Returns clear validation errors if the input is invalid

Validation

Starlette

```
data = await request.json()

if "name" not in data:
    return JSONResponse({"error": "Name required"}, status_code=400)
```

FastAPI

```
class Customer(BaseModel):
    name: str
```

If the client omits name, FastAPI automatically returns a **422 Unprocessable Entity** response describing the validation issue.

API Documentation

Starlette

No automatic documentation is provided.

FastAPI

Available immediately after starting the server:

- Swagger UI: http://localhost:8000/docs
- ReDoc: http://localhost:8000/redoc

No extra code is required.

Dependency Injection

Starlette

You typically create and pass dependencies manually.

FastAPI

```
from fastapi import Depends

def get_db():
    return "Database Connection"

@app.get("/")
```

```
async def home(db=Depends(get_db)):
    return {"db": db}
```

This is especially useful for databases, authentication, and shared services.

Middleware

Starlette

```
from starlette.middleware import Middleware
```

FastAPI

```
app.add_middleware(SomeMiddleware)
```

FastAPI uses Starlette's middleware system.

Performance

Both frameworks are built on Starlette and are extremely fast. In most real-world applications, performance differences are negligible.

When to Choose Starlette

Choose **Starlette** if you are building:

- A lightweight ASGI application
- Middleware
- WebSocket servers
- A custom framework
- A low-level async service where you want full control

When to Choose FastAPI

Choose **FastAPI** if you are building:

- REST APIs
- Microservices
- Backend services
- AI/ML model serving
- Enterprise APIs
- CRUD applications
- Projects that benefit from automatic validation and documentation

---

Advantages of FastAPI

- Automatic request validation
- Automatic OpenAPI generation
- Interactive Swagger UI
- Pydantic model integration
- Dependency injection
- Cleaner, less repetitive code
- Strong editor support through type hints

Advantages of Starlette

- Minimal and lightweight
- Greater control over the request lifecycle
- Excellent foundation for custom frameworks
- Lower overhead in simple applications

Recommendation

For most application development—especially CRUD APIs, microservices, and production REST services—**FastAPI** is the better choice because it builds on Starlette while providing automatic validation, documentation, and many productivity features.

Use **Starlette** when you specifically need a lightweight ASGI framework or are building infrastructure components where you prefer lower-level control over the HTTP layer.

Real-World Example

Imagine an endpoint that retrieves data from a database and calls an external API.

WSGI (synchronous):

1. Receive request.
2. Wait for the database.
3. Wait for the external API.
4. Send the response.

During the waiting periods, that worker cannot process another request.

ASGI (asynchronous):

1. Receive request.
2. Start the database operation.
3. While waiting, process other incoming requests.
4. Resume when the database responds.
5. Call the external API.
6. Again, process other requests while waiting.
7. Send the response when everything is complete.

This ability to make progress on other work while waiting is what allows ASGI applications to handle many concurrent connections efficiently.

Demo with CRUD using starlette server :

```
from starlette.applications import Starlette
from starlette.responses import JSONResponse
from starlette.requests import Request
from starlette.routing import Route

# In-memory database
customers = []



# GET /customers
async def get_customers(request: Request):
    return JSONResponse(customers)



# GET /customers/{id}
async def get_customer(request: Request):
    customer_id = int(request.path_params["id"])

    for customer in customers:
        if customer["id"] == customer_id:
            return JSONResponse(customer)

    return JSONResponse({"message": "Customer not found"}, status_code=404)
```

```
# POST /customers
async def create_customer(request: Request):
    data = await request.json()

    customer = {
        "id": len(customers) + 1,
        "name": data.get("name"),
        "email": data.get("email"),
        "city": data.get("city")
    }

    customers.append(customer)

    return JSONResponse(customer, status_code=201)



# PUT /customers/{id}
async def update_customer(request: Request):
    customer_id = int(request.path_params["id"])
    data = await request.json()

    for customer in customers:
        if customer["id"] == customer_id:
            customer["name"] = data.get("name", customer["name"])
            customer["email"] = data.get("email", customer["email"])
            customer["city"] = data.get("city", customer["city"])
```

```
            return JSONResponse(customer)

    return JSONResponse({"message": "Customer not found"}, status_code=404)


# DELETE /customers/{id}
async def delete_customer(request: Request):
    customer_id = int(request.path_params["id"])

    for customer in customers:
        if customer["id"] == customer_id:
            customers.remove(customer)
            return JSONResponse(
                {"message": "Customer deleted successfully"}
            )

    return JSONResponse({"message": "Customer not found"}, status_code=404)


routes = [
    Route("/customers", endpoint=get_customers, methods=["GET"]),
    Route("/customers/{id:int}", endpoint=get_customer, methods=["GET"]),
    Route("/customers", endpoint=create_customer, methods=["POST"]),
    Route("/customers/{id:int}", endpoint=update_customer, methods=["PUT"]),
    Route("/customers/{id:int}", endpoint=delete_customer, methods=["DELETE"]),
]
```

```
app = Starlette(debug=True, routes=routes)
```